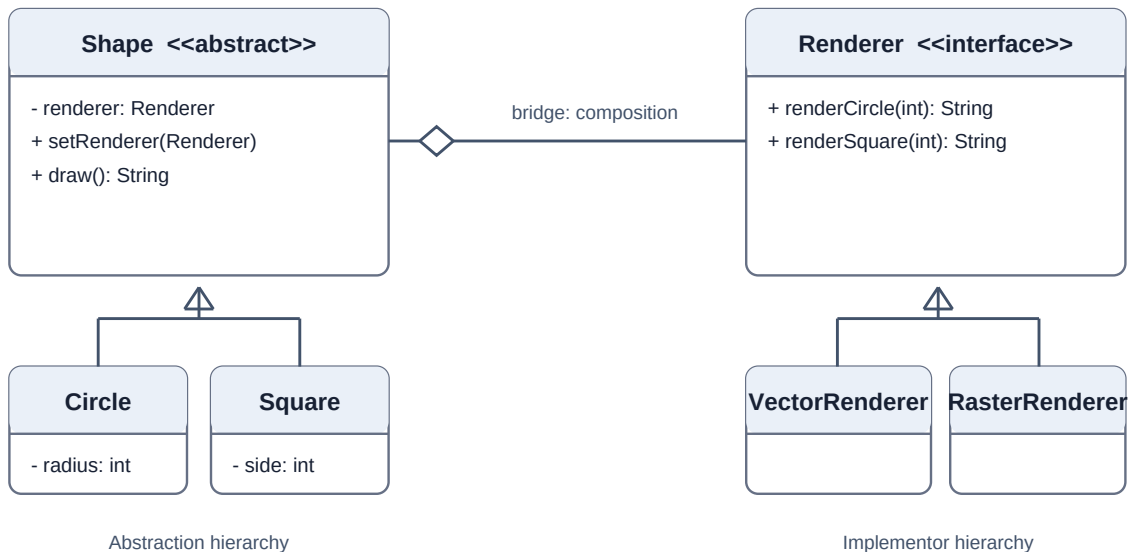


Assignment 3 Report: Bridge Pattern

1. Introduction

The chosen topic is Shape-Renderer. A shape describes *what* to draw, while a renderer decides *how* to represent it. The two dimensions can vary separately: a new shape can use any existing renderer, and a new renderer can work with every existing shape. Bridge fits because **Shape** contains a **Renderer** reference and delegates drawing through its interface.

2. UML class diagram



Shape is the abstraction; **Circle** and **Square** are refined abstractions. **Renderer** is the implementor; the vector and raster classes are concrete implementors. **BridgeDemo** is the client. It changes the renderer of the same shape objects at runtime. Bridge differs from Adapter: Adapter makes an existing incompatible interface usable, while Bridge is designed so two dimensions can evolve independently.

3. Clean Code principles

The before snippets show simpler but less maintainable design choices. The after snippets are excerpts from the submitted code.

1. Separation of responsibilities

Before, the shape chooses the output format itself:

```
if (raster) return drawPixels();
return drawSvg();
```

After, it delegates representation to a renderer:

```
public String draw() {
    return renderer().renderCircle(radius);
}
```

Shape data stays in Circle; output logic stays in renderer classes.

2. Meaningful names

Before, generic names hide the roles:

```
interface Output { String make(int value); }
```

After, names identify the operation and parameter:

```
interface Renderer {  
    String renderCircle(int radius);  
    String renderSquare(int side);  
}
```

Names such as Shape, Renderer, Circle, and VectorRenderer make the two hierarchies easy to recognize.

3. Avoid duplicate format selection

Before, every shape needs a format branch:

```
return raster ? rasterCircle(radius) : vectorCircle(radius);
```

After, the selected renderer supplies the behavior:

```
return renderer().renderCircle(radius);
```

The client selects a renderer once. Shape subclasses contain no repeated raster/vector switch.

4. Open to new renderers

Before, adding a format requires changing conditionals in the shape classes:

```
if (format.equals("raster")) { ... }  
else if (format.equals("vector")) { ... }
```

After, Shape depends only on an interface:

```
private Renderer renderer;
```

A third Renderer can be added without changing Shape, Circle, or Square. Every renderer must still support each operation declared by the interface.

5. Validate state at the boundary

Before, an invalid dimension could reach a renderer:

```
this.radius = radius;
```

After, the constructor rejects it immediately:

```
if (radius < 1) {  
    throw new IllegalArgumentException("radius must be positive");  
}  
this.radius = radius;
```

Shape.setRenderer also rejects null, so every constructed shape is ready to draw.

4. Conclusion

Bridge adds an interface and several small classes, which is more structure than a single drawing method. In return, shapes and renderers can be combined freely. The demo changes renderers at runtime without replacing shape objects. The main limitation of this simple interface is that adding a new shape type requires a new method in every renderer.

5. GitHub repository

<https://github.com/RamazanUtegen/sdp-bridge-pattern-assignment-3>